

Deploying a web application with Docker

Ornella Irrera

Department of Information Engineering, University of Padova

March 21st, 2024

Outline

- Web application development
- Containerized web applications
- Docker overview and main objects
- Containerize your web application projects

Developing a web application

Developing a web application requires the integration of several components which usually requires manual installation and configuration.

- To develop the **Data Layer** you relied on **postgreSQL**
- To develop the **Business Layer** you installed a specific **Java version** in your machine
- Then you used **Maven** to generate a war file with all the resources needed to deploy your web application. But Maven depends on **Java**
- The war file is finally deployed on **TOMCAT** which strictly depends on your **java** version



Developing a web application

As a consequence, having your web application up and running in your computer, is closely tied to how you configured each component.

Furthermore, all these components are interdependent; for instance, different versions of Tomcat may rely on different versions of Java.

Running and maintaining your web application in your computer is easy and you have the full control of the data, business and presentation layers.

Developing a web application

As a consequence, having your web application up and running in your computer, is closely tied to how you configured each component.

Furthermore, all these components are interdependent; for instance, different versions of Tomcat may rely on different versions of Java.

Running and maintaining your web application in your computer is easy and you have the full control of the data, business and presentation layers.

...But it is not always the case

Problem

What would happen if you have to deploy your web application on a new server that has different configurations, for example different PostgreSQL versions than yours?

Problem

What would happen if you have to deploy your web application on a new server that has different configurations, for example different PostgreSQL versions than yours?

In this situation it is usually impossible to faithfully re-implement the entire configuration of your machine. **Deploying your web application would be difficult.**

Additionally, you would need to **modify** your **code** and all your **dependencies** to ensure proper functioning of your web application in a different environment. This is extremely time demanding.

Problem

What would happen if you have to deploy your web application on a new server that has different configurations, for example different PostgreSQL versions than yours?

In this situation it is usually impossible to faithfully re-implement the entire configuration of your machine. **Deploying your web application would be difficult.**

Additionally, you would need to **modify** your **code** and all your **dependencies** to ensure proper functioning of your web application in a different environment. This is extremely time demanding.

You need a solution that allows you to easily maintain your code, and ensure a proper functioning also in these conditions without spending too much time in adapting your code to a different, dynamic environment.

Why Maven is not enough?

Maven allows you to managing java projects and ensures a standardized and consistent approach to building projects.

The build life cycle build process involves compiling source code, running tests, packaging the application, and managing dependencies according to the project configuration specified in the pom.xml file.

All the project resources, compiled classes, and dependencies, are packed in a WAR which can be deployed.

Why Maven is not enough?

Maven allows you to managing java projects and ensures a standardized and consistent approach to building projects.

The build life cycle build process involves compiling source code, running tests, packaging the application, and managing dependencies according to the project configuration specified in the pom.xml file.

All the project resources, compiled classes, and dependencies, are packed in a WAR which can be deployed.

However, the environment where you deploy your web application does not depend on Maven and each component (i.e., tomcat and postgres) has its versions and compatibility must be ensured.

Solution

A solution to overcome these problems is **containerization**.

Containerization of a web application is the process of packaging the application and its dependencies into a **container** → a lightweight, standalone executable unit.

Solution

A solution to overcome these problems is **containerization**.

Containerization of a web application is the process of packaging the application and its dependencies into a **container** → a lightweight, standalone executable unit.

Containers provide an **isolated runtime environment** that encapsulates all the necessary components, such as the application code, runtime libraries, system tools, and configuration files.

Solution

A solution to overcome these problems is **containerization**.

Containerization of a web application is the process of packaging the application and its dependencies into a **container** → a lightweight, standalone executable unit.

Containers provide an **isolated runtime environment** that encapsulates all the necessary components, such as the application code, runtime libraries, system tools, and configuration files.

Containerization allows the web application to be deployed and run consistently across different environments, **regardless of the underlying infrastructure or operating system**. The container ensures that the application runs consistently, without conflicts or dependencies on the host system.

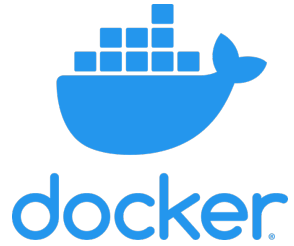
Solution

This is useful not only when you have to deploy a web application, but in a wide range of use cases. For example:

- You have Python3.8 installed in your machine and you need to run some code developed for python3.6;
- Your OS is Windows and you want to use some tools which can be installed exclusively on Linux;

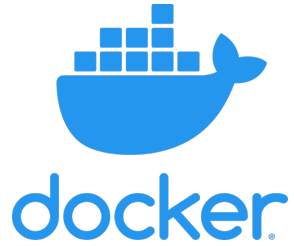
Containerization creates isolated environments independent of your host machine.

Docker



Docker is an **open platform** for **developing, shipping, and running applications**. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly.

Docker



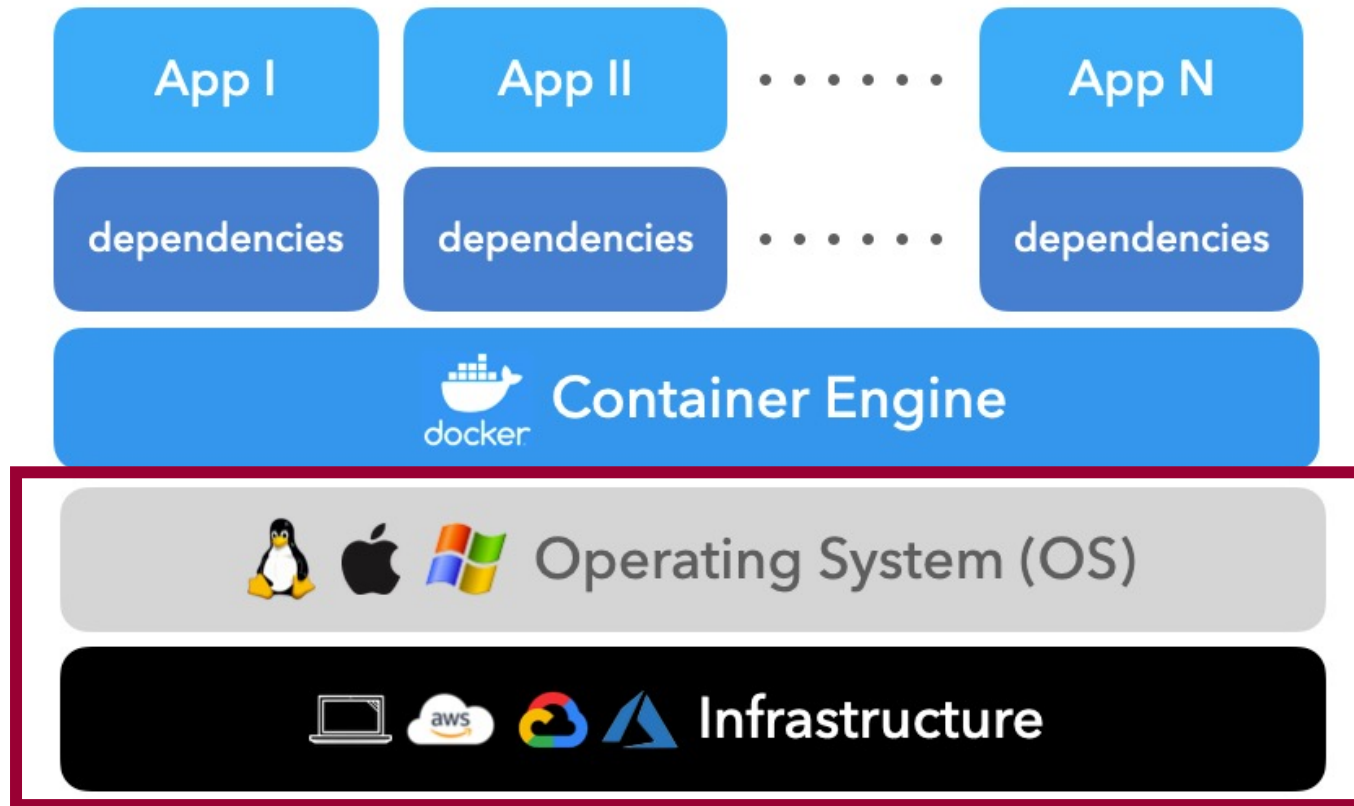
Docker is an **open platform for developing, shipping, and running applications**. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly.

Docker provides the ability to package and run an application in a isolated environment called a **container**.

The isolation and security allows you to **run many containers simultaneously** on a given **host**.

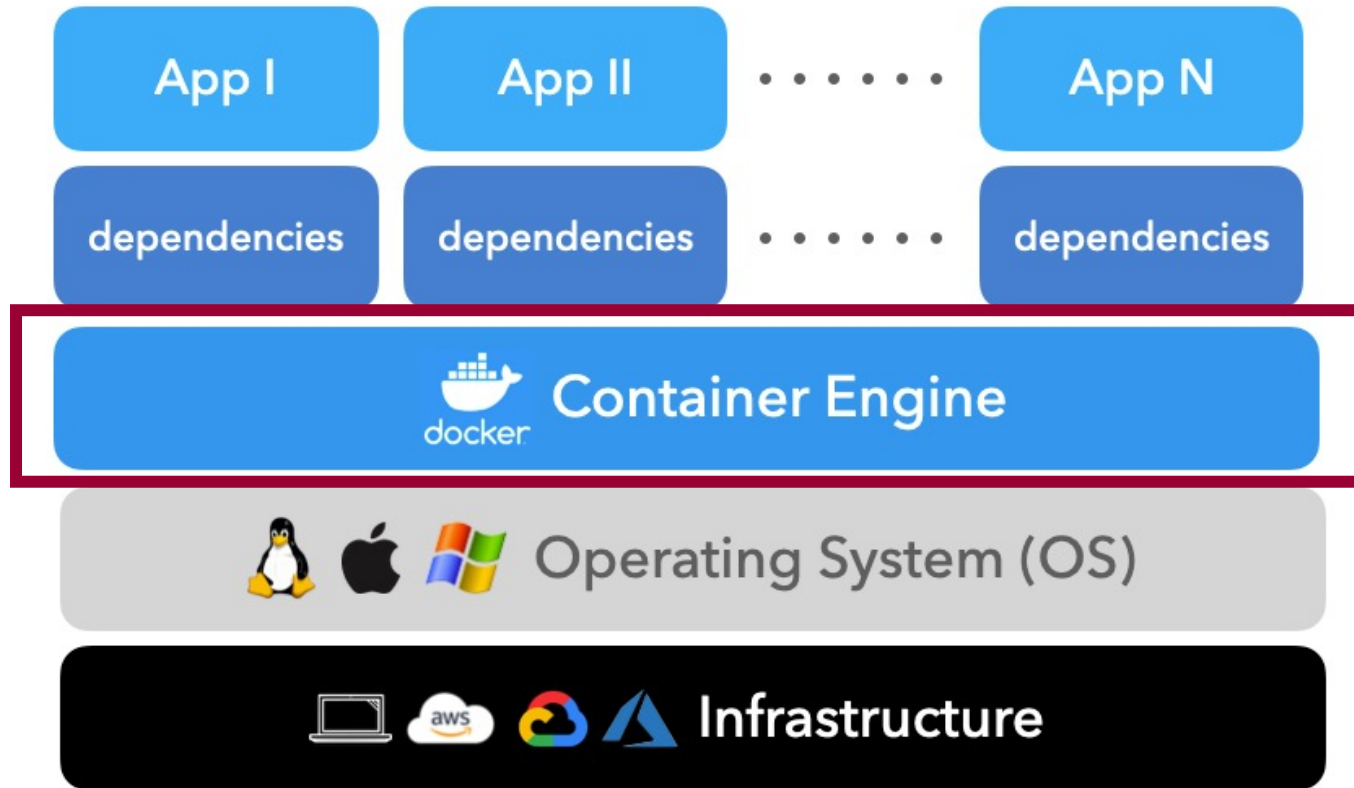
Containers are lightweight and contain everything needed to run the application, **so you do not need to rely on what is currently installed on the host**.

Docker – An overview



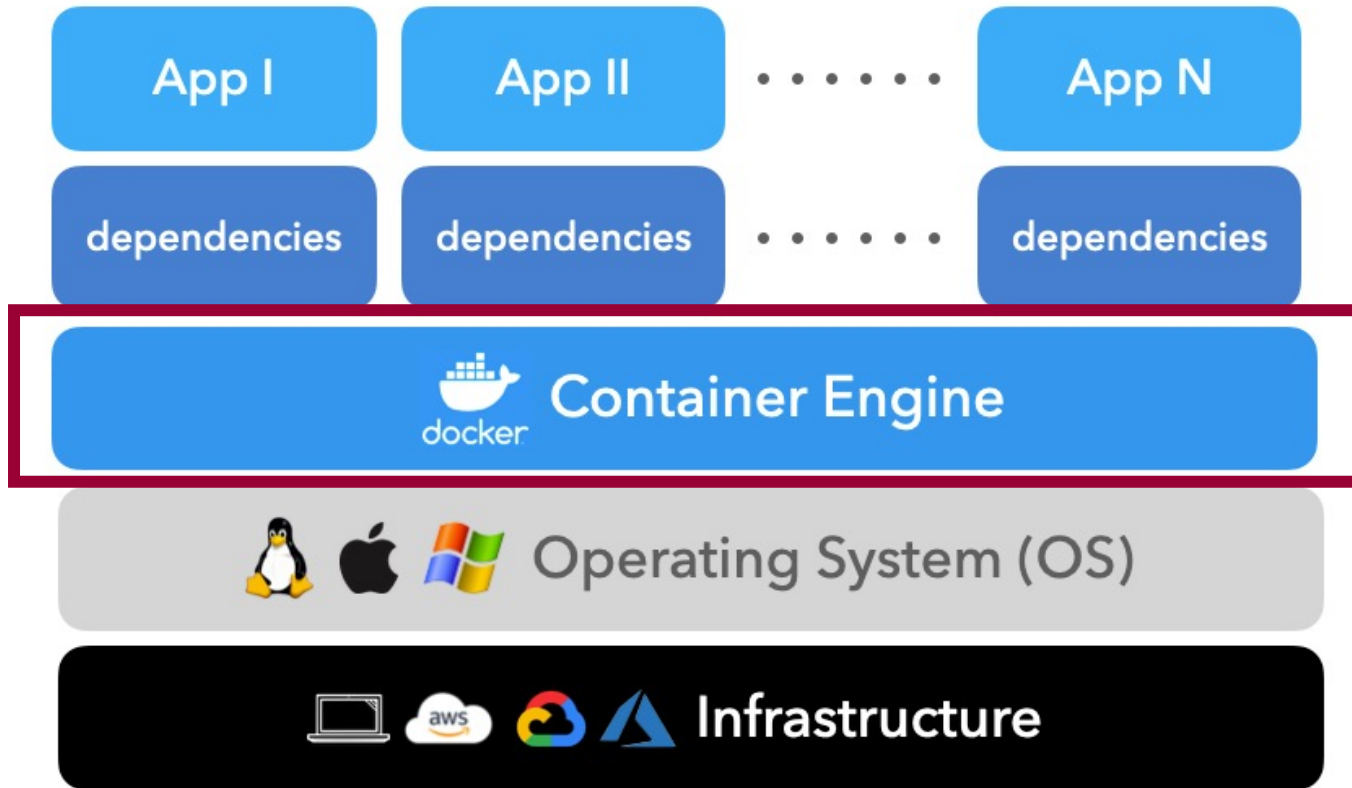
Where docker runs. It is the host server which typically has its own Operating system.

Docker – An overview



A **container engine**, is a software component responsible for running and managing containers on a host machine.

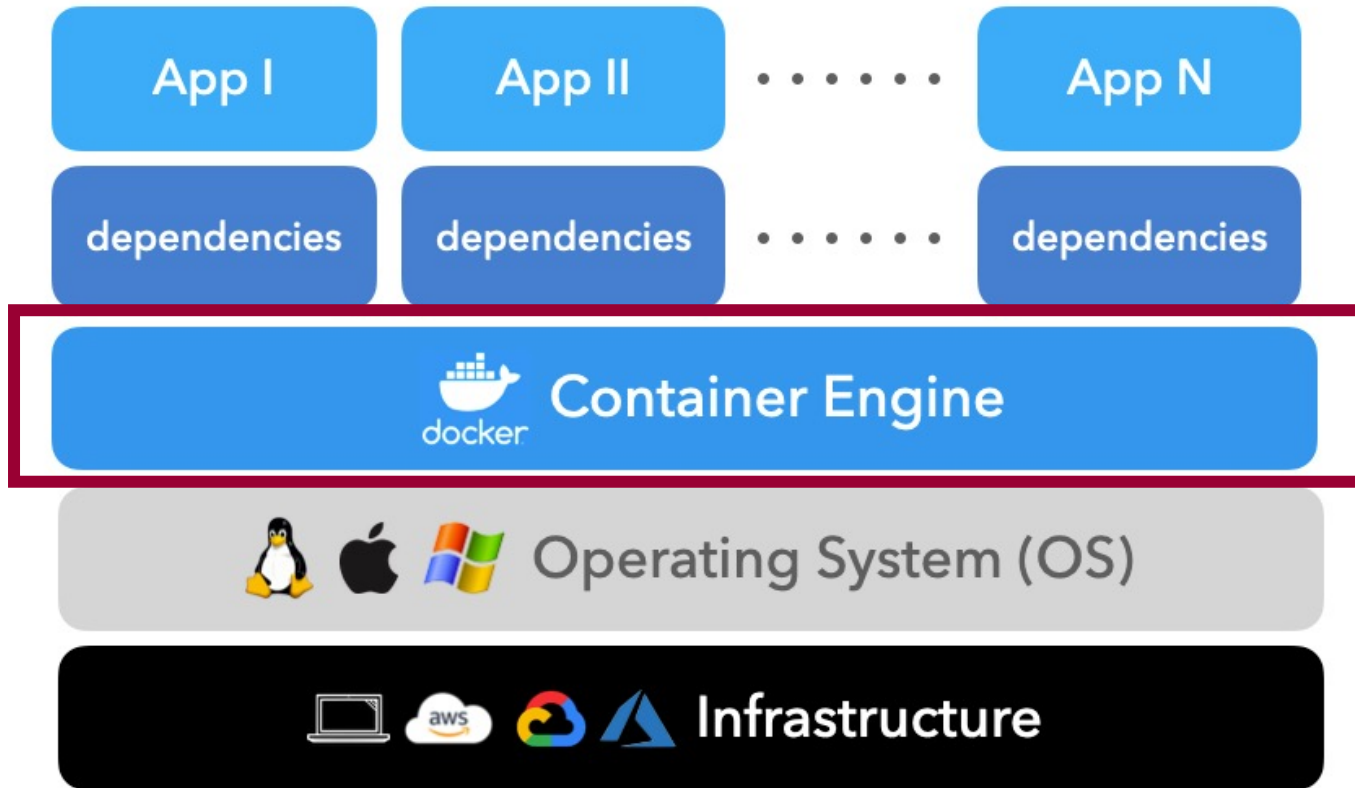
Docker – An overview



A **container engine**, is a software component responsible for running and managing containers on a host machine.

It provides the necessary functionality to **create, start, stop, and manage containers**, allowing applications to run in isolated environments.

Docker – An overview

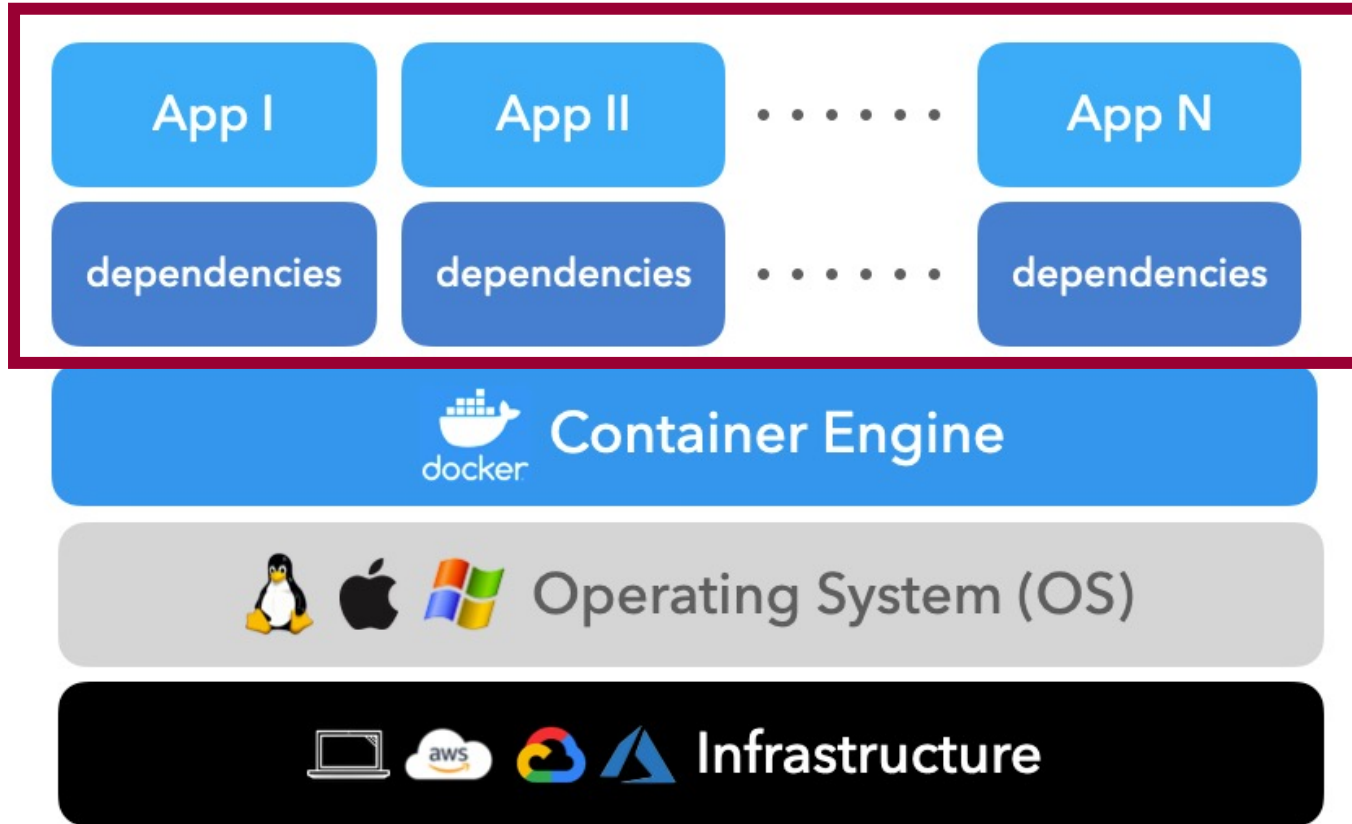


A **container engine**, is a software component responsible for running and managing containers on a host machine.

It provides the necessary functionality to create, start, stop, and manage containers, allowing applications to run in isolated environments.

Docker Engine is one of the most popular container engines. It manages the container lifecycle, networking, storage, and other essential aspects.

Docker – An overview



Isolated containerized applications. Each application has its own dependencies, its own libraries and are **independent** of the underlying infrastructure and OS.

Docker Advantages

Docker enhances:

- **PORTABILITY** → Docker containers can be easily packaged, distributed, and run on any system that supports Docker, regardless of the underlying infrastructure or operating system.

Docker Advantages

Docker enhances:

- **EFFICIENCY** → Docker containers are lightweight and share the host system's resources, resulting in efficient resource utilization. Multiple containers can run on a single host machine, enabling higher density and efficient utilization of computing resources.

Docker Advantages

Docker enhances:

- **SCALABILITY** → Docker allows applications to be deployed as multiple containers. Containers can be easily scaled up or down based on demand, providing the ability to handle increased workload.

Docker Advantages

Docker enhances:

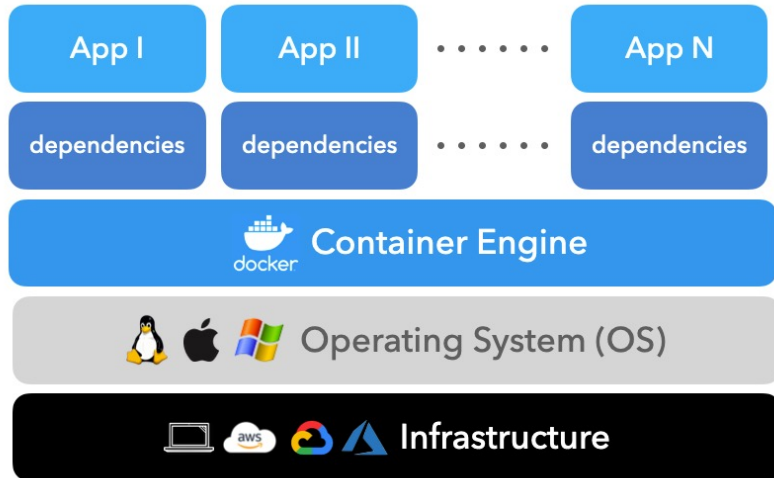
- **ISOLATION** → Docker containers provide process-level isolation, ensuring that applications and their dependencies run in isolated environments without interfering with each other or the host system.

Docker Advantages

Docker enhances:

- **PORTABILITY** → Docker containers can be easily packaged, distributed, and run on any system that supports Docker, regardless of the underlying infrastructure or operating system.
- **EFFICIENCY** → Docker containers are lightweight and share the host system's resources, resulting in efficient resource utilization. Multiple containers can run on a single host machine, enabling higher density and efficient utilization of computing resources.
- **SCALABILITY** → Docker allows applications to be deployed as multiple containers. Containers can be easily scaled up or down based on demand, providing the ability to handle increased workload.
- **ISOLATION** → Docker containers provide process-level isolation, ensuring that applications and their dependencies run in isolated environments without interfering with each other or the host system.

Containers vs Virtual Machine



Containers

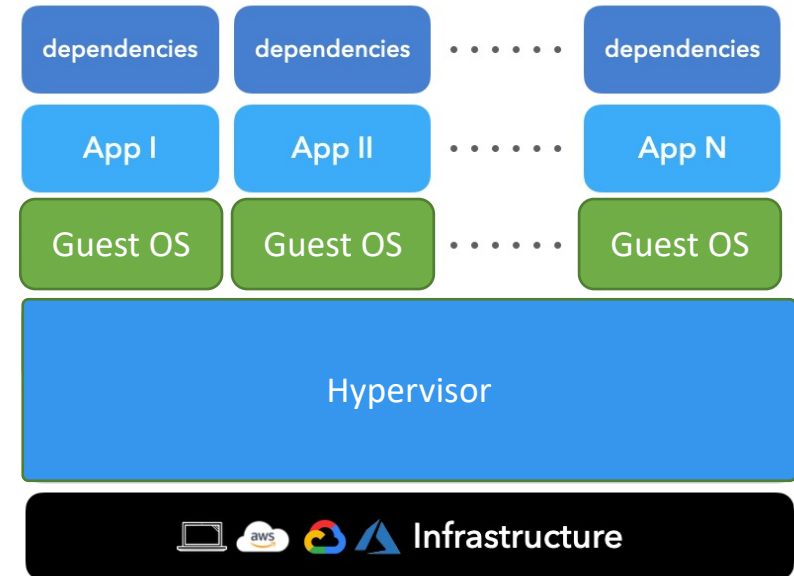
Containers main features:

- Portable
- Lightweight
- Fast
- Easy deployment

But:

- They all share the same kernel!

VS



VM

VMs main features:

- More isolated
- But:
- Slower
 - Difficult to be replicated
 - Resource intensive

Docker – (the most important) objects

The Docker objects collaborate together to run, access and manage the applications containers. The essential objects are the following:

- Images
- Dockerfiles
- Containers
- Volumes
- Services

Other objects are: plugins, networks, and registries

Docker Image

A *Docker image* is a read-only template with instructions for creating a **container**. It contains the code, runtime environment information, libraries, dependencies, and configurations.

A Docker image is a snapshot of the libraries and dependencies required by a container to run properly.

Users can create their own images or use those created by others and published in a **registry**.

The **build** process allows the image to be executed, downloading and installing all the dependencies and code.

Dockerfiles

To build user-defined images, users must create a *Dockerfile* with a simple syntax for defining the steps needed to create the image and run it.

Dockerfiles usually contain information about the **dependencies** – e.g., **requirements.txt** file, the **code directory**, **port exposure**, and **commands to run the application** or the *base image*.

Each instruction in a Dockerfile creates a layer in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt.

Docker Containers

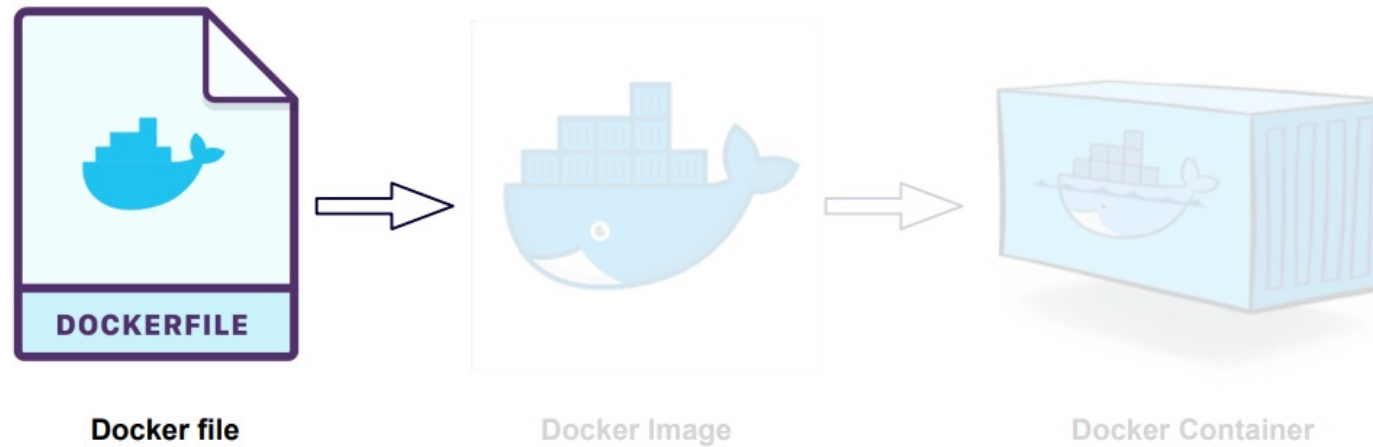
A Docker container is a standalone, lightweight, and isolated runtime environments.

Containers need a runnable image to exist. Containers are dependent on images: images are used to generate runtime environments and are needed to run an application.

It is an instance of a Docker image, created and managed by the Docker engine.

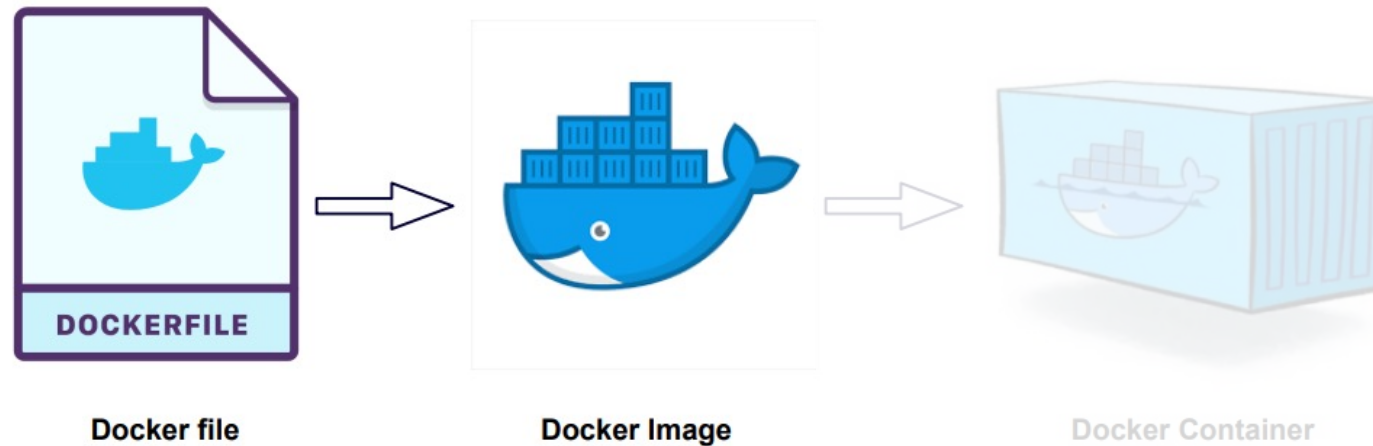
Multiple containers relying on the same image are allowed to run on the same host without interfering with each other or the underlying system.

Recap – dockerfiles, images, containers



A Docker image is built based on instructions defined in a Dockerfile, which specifies the steps to create the image.

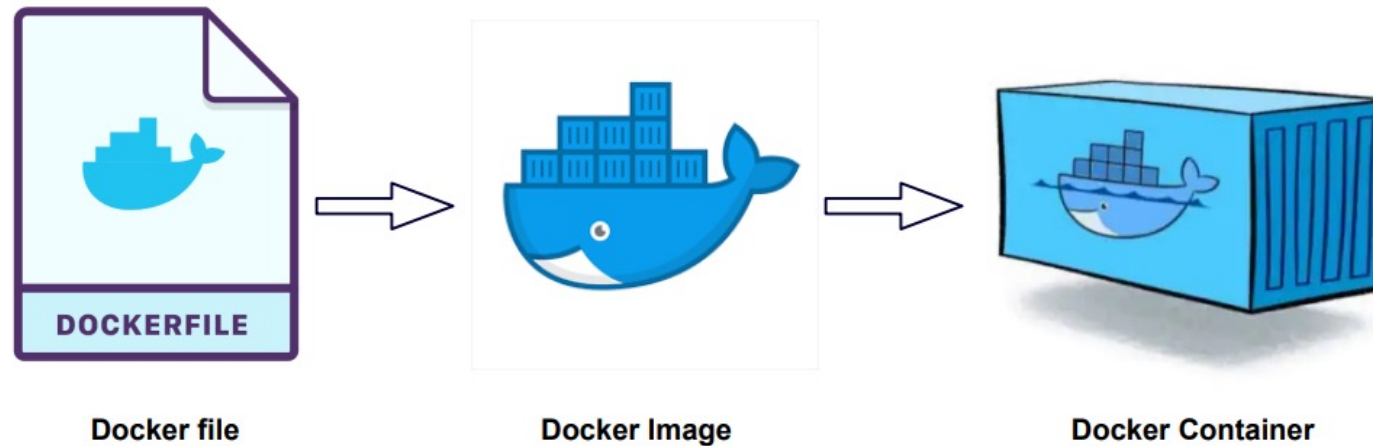
Recap – dockerfiles, images, containers



A Docker image is built based on instructions defined in a Dockerfile, **which specifies the steps to create the image.**

Images are created using the Docker **build** process, where the Docker Engine reads the Dockerfile and executes the instructions to build the image layer by layer. Each layer represents a specific step in the build process.

Recap – dockerfiles, images, containers

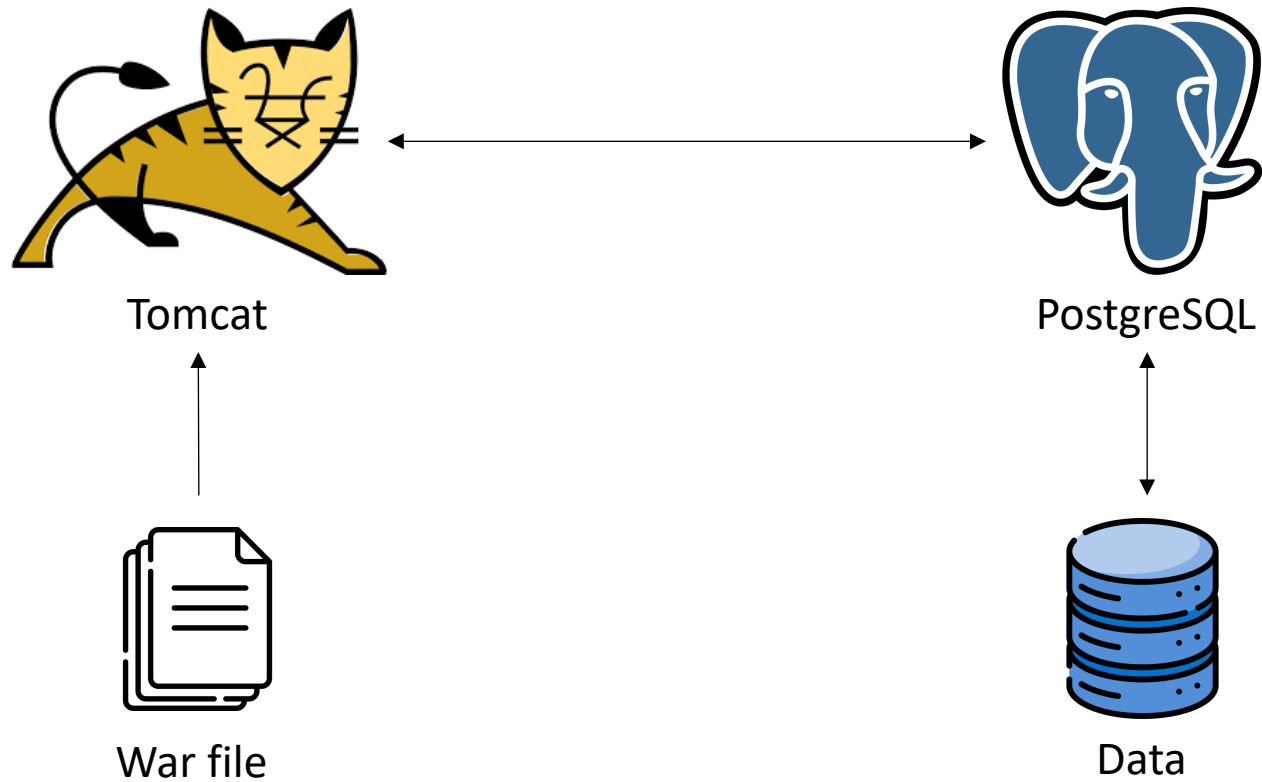


A Docker image is built based on instructions defined in a Dockerfile, **which specifies the steps to create the image**.

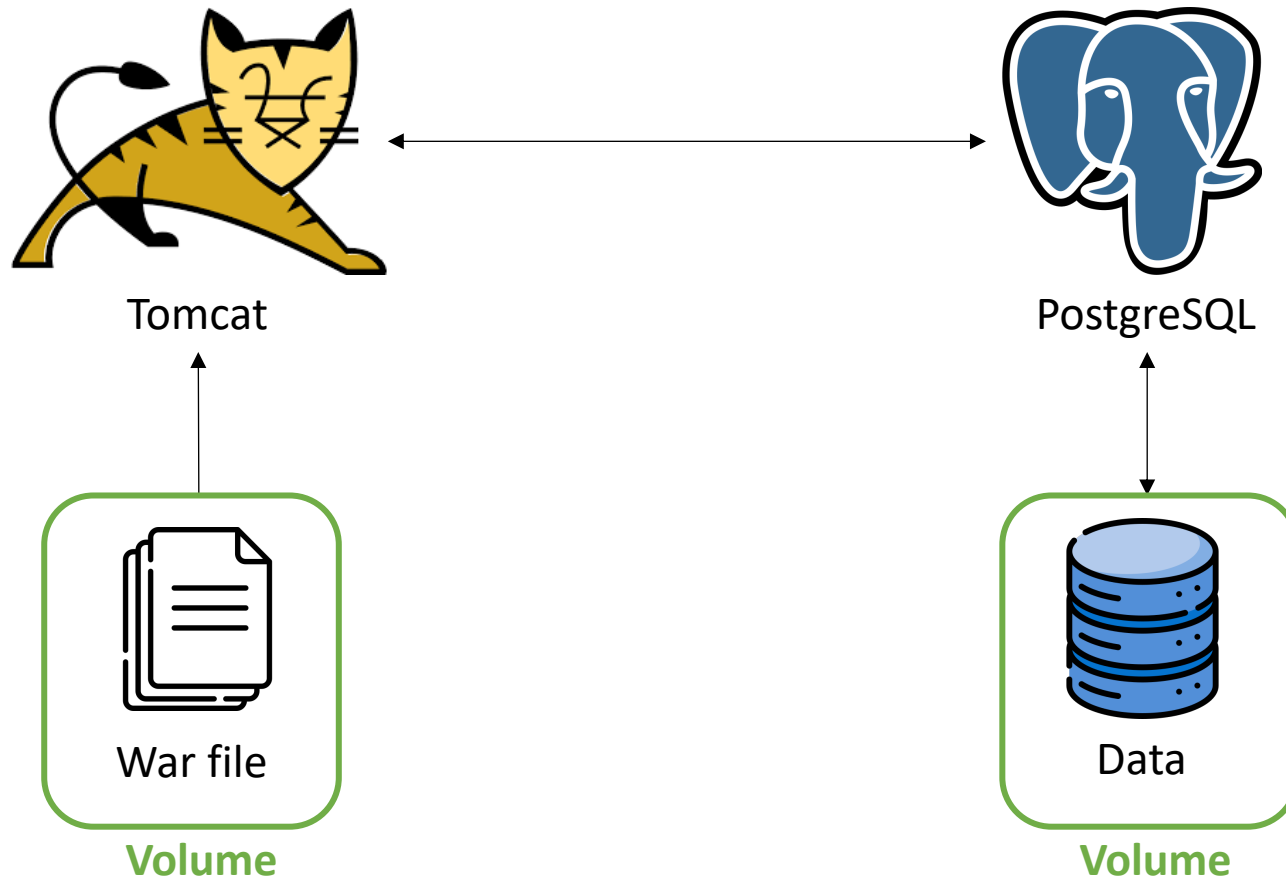
Images are created using the Docker build process, where the Docker Engine reads the Dockerfile and executes the instructions to build the image layer by layer. Each layer represents a specific step in the build process.

When a Docker image is built, it is possible to create an instance of it called a **container**, which is an isolated and lightweight runtime environment that executes the application using the resources of the host machine. Multiple containers can be created and run from the same image, each with its own isolated execution environment and state.

Containerize a real web application



Containerize a real web application



Docker Volumes

In Docker, a **volume** is a mechanism for persistently storing data generated by and accessed by the docker containers. A volume provides a way to share and manage data between the host machine and the containers.

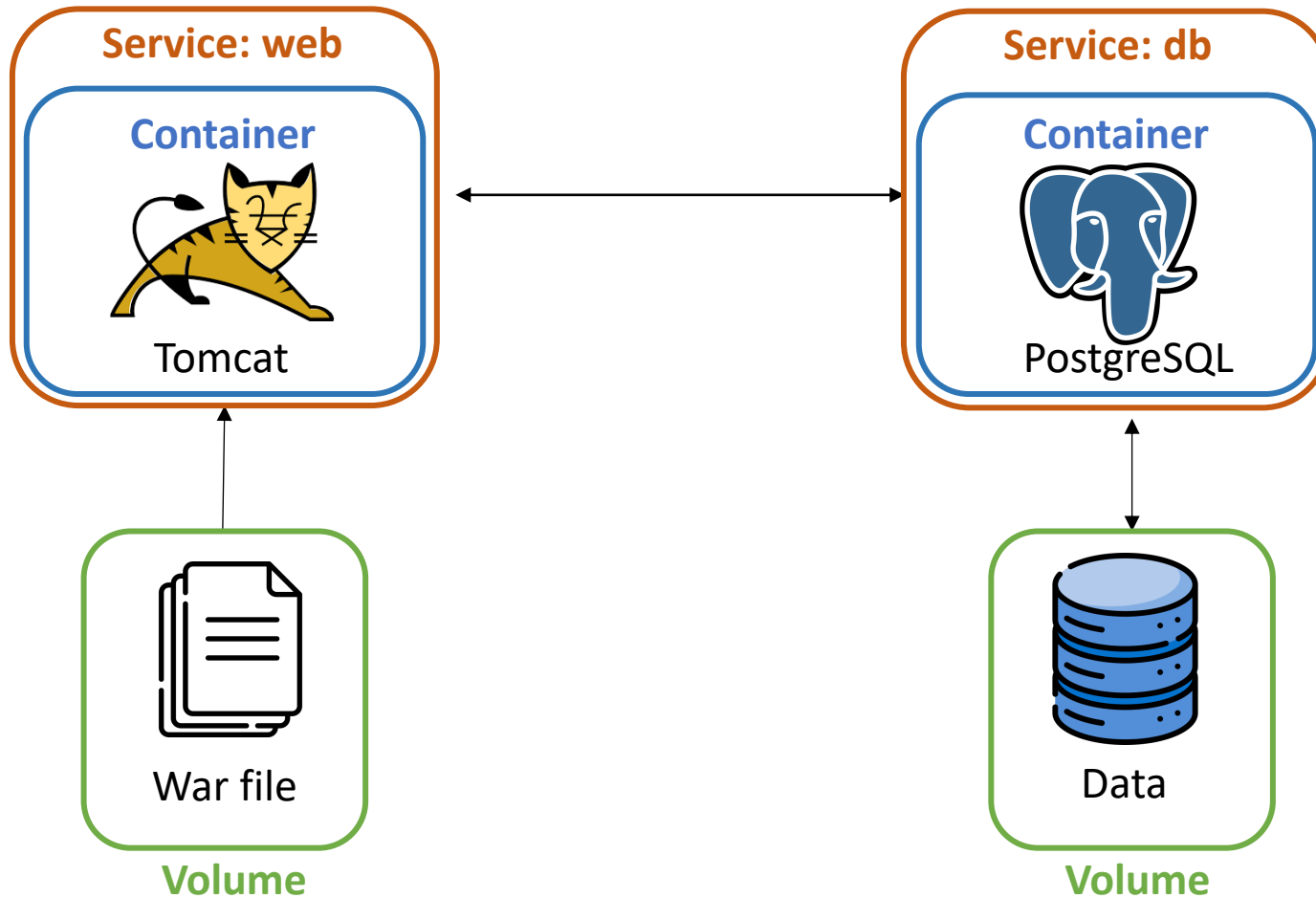
A Docker volume is a directory or file stored outside of the container's writable layer. It can be used to store **databases**, **configuration files**, **logs**, and any other data that needs to persist beyond the lifecycle of a container.

Docker Volumes

Important concepts about volumes:

- Multiple containers can mount the same volume, allowing them to access and modify shared data;
- Volumes allow data to persist even if a container is stopped or deleted;
- Volumes are not tied to one or more specific containers. They can be attached and detached from containers as needed.

Containerize a real web application



Docker Services

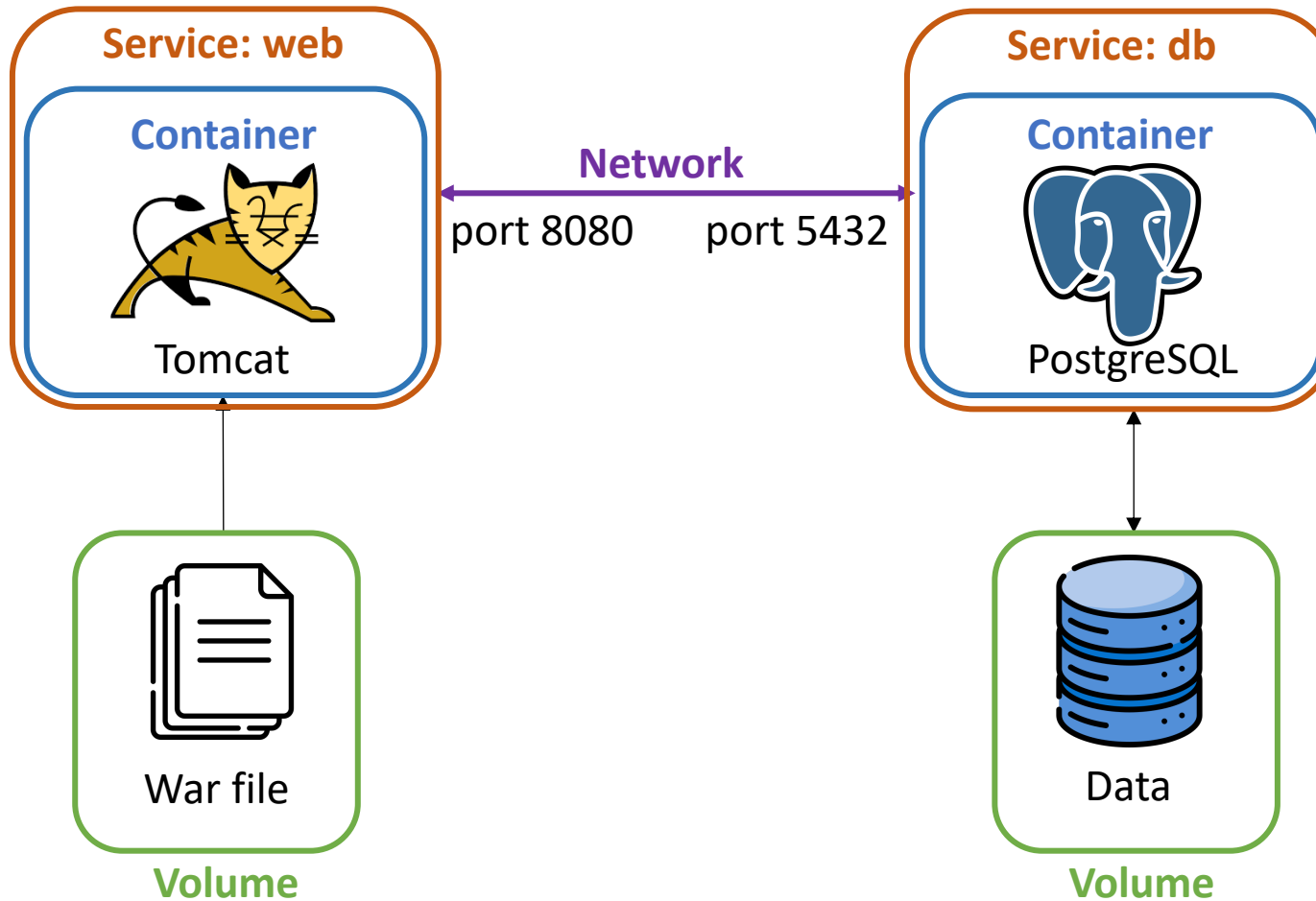
A service in Docker Compose represents a component of an application.

The definition of a service includes also the information about the the Docker image, environment variables, ports, volumes, and networks.

Each service runs one type of image; each service might run multiple containers, all performing the same function.

Services allow you to manage and deploy different components of your application as separate entities, establishing communication and dependencies between them.

Containerize a real web application



Docker Networks

Docker containers are all isolated and they cannot communicate: hence, given two containers, A and B, exposing different ports, A cannot communicate with B.

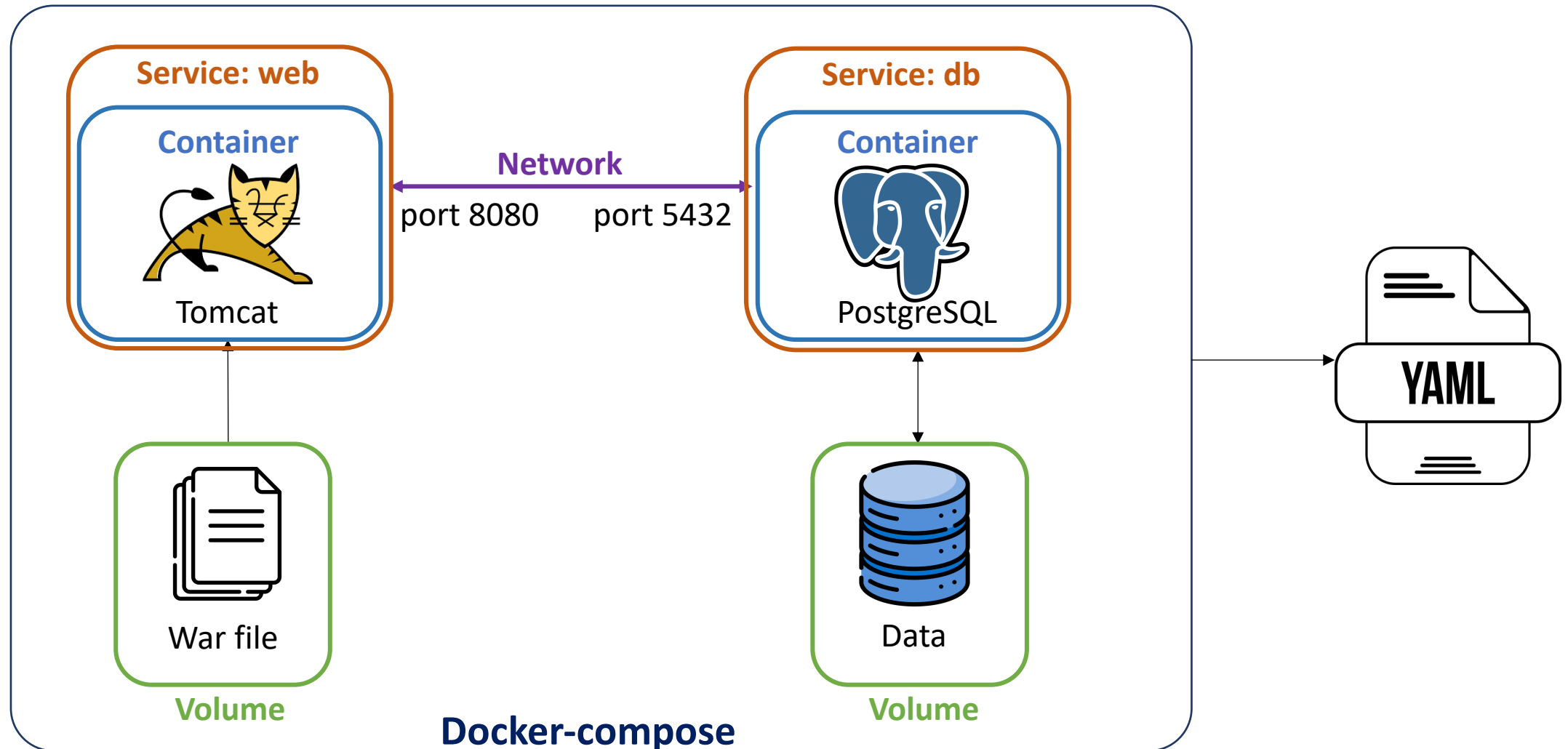
On the other hand, we can reach A and B from outside the container, if the ports are properly specified.

Docker networks are the solution to let containers communicate.

Only the containers belonging to the same network can communicate.

When a container is instantiated, the network it belongs to should be specified.

Containerize a real web application



Docker Compose

Docker Compose is a tool that simplifies the management and deployment of multi-containers applications.

It allows you to define and configure multiple Docker **services**, **networks** and **volumes** as a single application using a YML file.

Typical use-case: you have a web application. Each of these component is treated as a separate container which is part of the same web application.

Summing up: Containerize a web application

We developed our entire web application which includes the backend developed in java, the frontend developed in css, html and js, and a postgresSQL database.

Maven produces a **.war** file which can be deployed on a web server (**Tomcat**).

We want to deploy our web application on **multi-containerized** environment which implements two different **services**:

- **Tomcat**
- **PostgreSQL**

This allows you to deploy your war file you created locally on the version of tomcat and postgresSQL you prefer, without having to adapt your web application to the hosting infrastructure.

Containerize a web application step by step

All the services in a multi-containerized environment are described in the **docker-compose.yml** file. Docker Compose creates a network for your application, and each service is connected to that network. Services can communicate with each other using their service names as hostnames.

- We need to understand which are the services we need to deploy our web application: in this case we have *tomcat* and *PostgreSQL*.
- We need to specify some **dependencies**: establishing a hierarchies of dependencies allows to determine the order in which the services are executed.
- Each service relies on a specific **image, ports, volumes, environment variables**

Install Docker

Docker and docker compose can be installed on Windows, iOS, Linux. However, installing Docker on Windows might be difficult.

Docker → <https://docs.docker.com/get-docker/>

Docker compose → <https://docs.docker.com/compose/>

services:

web:

image: tomcat:10

ports:

– "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

– ./crane.war:/usr/local/tomcat/webapps/crane.war

db:

image: postgres

ports:

– "5432:5432"

environment:

– POSTGRES_PASSWORD=postgres

– POSTGRES_USER=postgres

volumes:

– ./crane.sql:/docker-entrypoint-initdb.d/init.sql

– ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50

services:

← In the docker-compose.yml file we define the services needed to run our web application.

```
web:
  image: tomcat:10
  ports:
    - "8080:8080"
  depends_on:
    db:
      condition: service_healthy
  volumes:
    - ./crane.war:/usr/local/tomcat/webapps/crane.war

db:
  image: postgres
  ports:
    - "5432:5432"
  environment:
    - POSTGRES_PASSWORD=postgres
    - POSTGRES_USER=postgres
  volumes:
    - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
    - ./data/db:/var/lib/postgresql/data
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U postgres" ]
    interval: 5s
    timeout: 10s
    retries: 50
```

services:

web:

```
image: tomcat:10
ports:
  - "8080:8080"
depends_on:
  db:
    condition: service_healthy
volumes:
  - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

db:

```
image: postgres
ports:
  - "5432:5432"
environment:
  - POSTGRES_PASSWORD=postgres
  - POSTGRES_USER=postgres
volumes:
  - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
  - ./data/db:/var/lib/postgresql/data
healthcheck:
  test: [ "CMD-SHELL", "pg_isready -U postgres" ]
  interval: 5s
  timeout: 10s
  retries: 50
```

web and *db* are the names of the two services we are defining. These names are useful not only to identify the individual services, but also to **establish connections and let the services communicate.**

services:

web:

```
image: tomcat:10
ports:
  - "8080:8080"
depends_on:
  db:
    condition: service_healthy
volumes:
  - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

db:

```
image: postgres
ports:
  - "5432:5432"
environment:
  - POSTGRES_PASSWORD=postgres
  - POSTGRES_USER=postgres
volumes:
  - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
  - ./data/db:/var/lib/postgresql/data
healthcheck:
  test: [ "CMD-SHELL", "pg_isready -U postgres" ]
  interval: 5s
  timeout: 10s
  retries: 50
```

Web and *db* run two containers, hence, we need to specify the image they refer to.

The image can be found on a registry (as in this case), and we need only to specify the image name; alternatively, it might be defined in a dockerfile and we have to specify the location of the dockerfile.

services:

```
web:
  image: tomcat:10
  ports:
    - "8080:8080"
  depends_on:
    db:
      condition: service_healthy
  volumes:
    - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

```
db:
  image: postgres
  ports:
    - "5432:5432"
  environment:
    - POSTGRES_PASSWORD=postgres
    - POSTGRES_USER=postgres
  volumes:
    - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
    - ./data/db:/var/lib/postgresql/data
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U postgres" ]
    interval: 5s
    timeout: 10s
    retries: 50
```

This option allows us to specify the port mapping: in this case, we have **host_port : container_port**

In order to access to each service within the container :

- The **hostname** is the name of the service
 - The port is the **container_port**
- <http://web:8080/> from the postgres service container

To access to each service outside the container:

- The **hostname** is **localhost**
 - The **port** is the **host_port**
- <http://localhost:8080/> from outside the container

You can check if a service is up and running with **curl**

services:

web:

```
image: tomcat:10
ports:
  - "8080:8080"
depends_on:
  db:
    condition: service_healthy
volumes:
  - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

db:

```
image: postgres
ports:
  - "5432:5432"
environment:
  - POSTGRES_PASSWORD=postgres
  - POSTGRES_USER=postgres
volumes:
  - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
  - ./data/db:/var/lib/postgresql/data
healthcheck:
  test: [ "CMD-SHELL", "pg_isready -U postgres" ]
  interval: 5s
  timeout: 10s
  retries: 50
```

A service allows us to specify environment variables, in this case we specify the username and the password we need to use to access the database.

These information are those you use also in your context.xml file

services:

```
web:
  image: tomcat:10
  ports:
    - "8080:8080"
  depends_on:
    db:
      condition: service_healthy
  volumes:
    - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

```
db:
  image: postgres
  ports:
    - "5432:5432"
  environment:
    - POSTGRES_PASSWORD=postgres
    - POSTGRES_USER=postgres
  volumes:
    - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
    - ./data/db:/var/lib/postgresql/data
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U postgres" ]
    interval: 5s
    timeout: 10s
    retries: 50
```

In order to define volumes we need to specify the (relative) path to the file in the hosting machine. After the «:» we specify where this file will be mounted inside the container.

The crane.sql file contains the code to create the database. The tables and the data will be automatically created as soon as the container is instantiated for the first time.

The **docker-entrypoint-init-db.d** directory is a convention to provide a mechanism for initializing a database when a container is first started.

services:

web:

```
image: tomcat:10
ports:
  - "8080:8080"
depends_on:
  db:
    condition: service_healthy
volumes:
  - ./crane.war:/usr/local/tomcat/webapps/crane.war
```

db:

```
image: postgres
ports:
  - "5432:5432"
environment:
  - POSTGRES_PASSWORD=postgres
  - POSTGRES_USER=postgres
volumes:
  - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
  - ./data/db:/var/lib/postgresql/data
```

healthcheck:

```
test: [ "CMD-SHELL", "pg_isready -U postgres" ]
interval: 5s
timeout: 10s
retries: 50
```

Healthchecks allow us to define commands to check the status of a service.

test → the command to execute. In this case we want to check the connection status of a postgresQL server. Without this check we would try to establish the connection when postgres is not ready and this would lead to connection refused.

Interval → interval at which the test is executed

Timeout → max amount of time the check has to finish

Retries → number of consecutive failures allowed

services:

web:

image: tomcat:10

ports:

- "8080:8080"

depends_on:

db:

condition: service_healthy

volumes:

- ./crane.war:/usr/local/tomcat/webapps/crane.war

db:

image: postgres

ports:

- "5432:5432"

environment:

- POSTGRES_PASSWORD=postgres

- POSTGRES_USER=postgres

volumes:

- ./crane.sql:/docker-entrypoint-initdb.d/init.sql

- ./data/db:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 10s

retries: 50

Dependencies establish the execution order. In this example, the **db** is initiated first; then, the **web** service is run as soon as the **healthcheck** is satisfied.

depends_on in general does not wait a service to be fully initialized and healthy. It only waits for the dependent service to be running. To wait a service to be ready we need to establish healthchecks.


```
ornellairrera@MacBook-Pro-di-Ornella docker % docker-compose up
```

```
[+] Running 2/0
```

```
✓ Container docker-db-1 Created
```

```
✓ Container docker-web-1 Created
```

```
0.0s
```

```
0.0s
```

```
Attaching to docker-db-1, docker-web-1
```

```
docker-db-1 | The files belonging to this database system will be owned by user "postgres".  
docker-db-1 | This user must also own the server process.
```

Starts with db service and creates the database

```
docker-db-1 | The database cluster will be initialized with locale "en_US.utf8".
```

```
docker-db-1 | The default database encoding has accordingly been set to "UTF8".
```

```
docker-db-1 | The default text search configuration will be set to "english".
```

```
docker-db-1 | Data page checksums are disabled.
```

```
docker-db-1 | fixing permissions on existing directory /var/lib/postgresql/data ... ok  
docker-db-1 | creating subdirectories ... ok
```

```
docker-db-1 | selecting dynamic shared memory implementation ... posix  
docker-db-1 | selecting default max_connections ... 100
```

```
docker-db-1 | selecting default shared_buffers ... 128MB  
docker-db-1 | selecting default time zone ... Etc/UTC
```

```
docker-db-1 | creating configuration files ... ok  
docker-db-1 | running bootstrap script ... ok
```

```
docker-db-1 | performing post-bootstrap initialization ... ok  
docker-db-1 | initdb: warning: enabling "trust" authentication for local connections
```

```
docker-db-1 | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.  
b.
```

```
docker-db-1 | syncing data to disk ... ok
```

```
docker-db-1 | Success. You can now start the database server using:  
docker-db-1 |  
docker-db-1 | pg_ctl -D /var/lib/postgresql/data -l logfile start
```

```
docker-db-1 | waiting for server to start...2023-05-30 09:33:07.039 UTC [84] LOG: starting PostgreSQL 15.3 (Debian 15.3-1.pgdg110+1) on x86_64-pc-linux-g  
nu, compiled by gcc (Debian 10.2.1-6) 10.2.1 20210110, 64-bit
```

```
docker-db-1 | 2023-05-30 09:33:07.042 UTC [84] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
```

```
docker-db-1 | 2023-05-30 09:33:07.083 UTC [87] LOG: database system was shut down at 2023-05-30 09:33:03 UTC
```

```
docker-db-1 | 2023-05-30 09:33:07.137 UTC [84] LOG: database system is ready to accept connections  
done
```

```
docker-db-1 | server started
```

```
docker-db-1 | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/init.sql  
docker-db-1 | CREATE DATABASE
```

```
docker-db-1 | GRANT  
docker-db-1 | You are now connected to database "ssadb" as user "postgres".
```

```
docker-db-1 | SET
```

```
docker-db-1 | SET
```

```
docker-db-1 | SET
```

```
docker-db-1 | SET
```

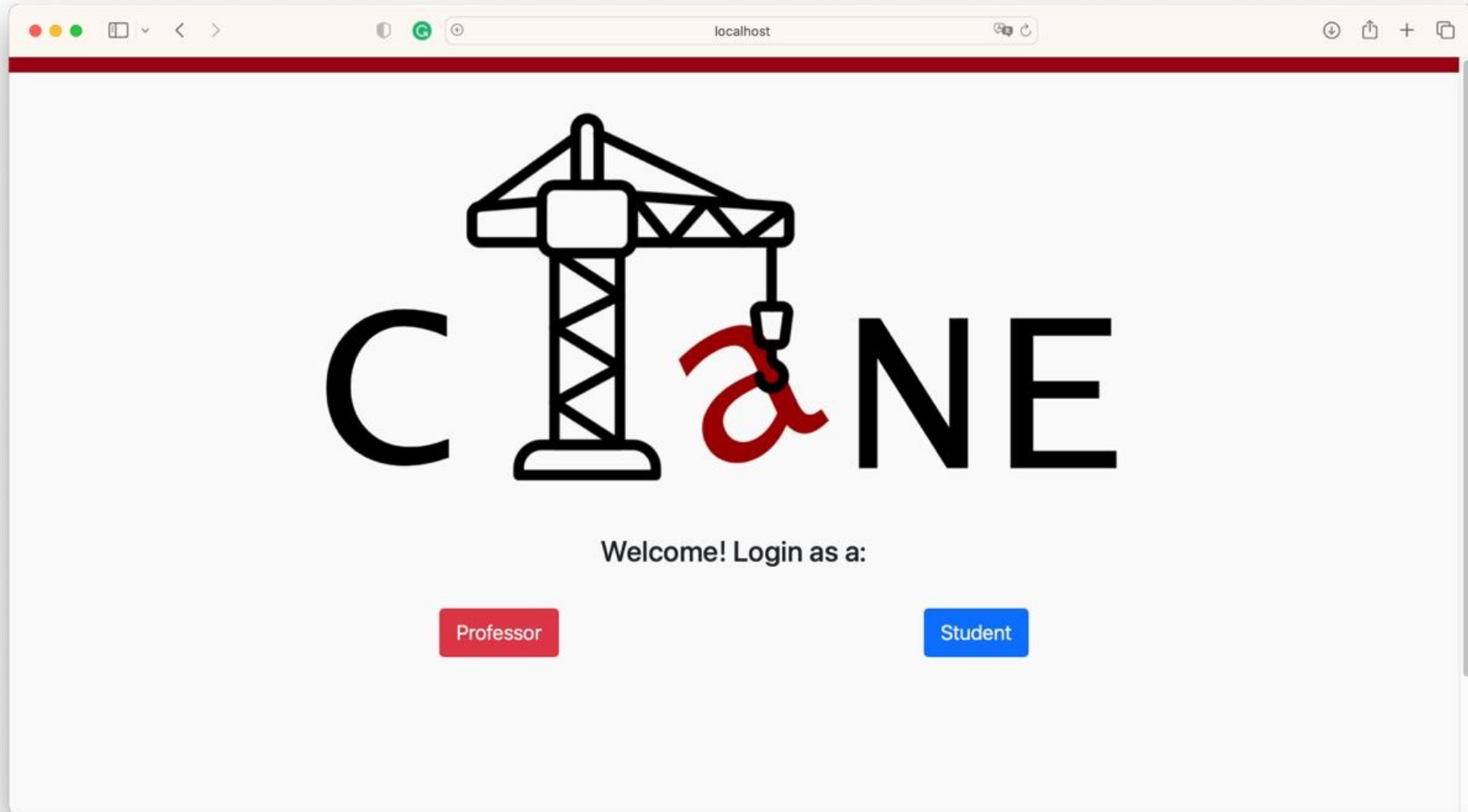
```
docker-db-1 | SET
```

```

docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 | ALTER TABLE
docker-db-1 |
docker-db-1 | 2023-05-30 09:33:12.796 UTC [84] LOG:  received fast shutdown request
docker-db-1 | waiting for server to shut down...2023-05-30 09:33:12.800 UTC [84] LOG:  aborting any active transactions
docker-db-1 | 2023-05-30 09:33:12.819 UTC [84] LOG:  background worker "logical replication launcher" (PID 90) exited with exit code 1
docker-db-1 | 2023-05-30 09:33:12.820 UTC [85] LOG:  shutting down
docker-db-1 | 2023-05-30 09:33:12.825 UTC [85] LOG:  checkpoint starting: shutdown immediate
docker-db-1 | .2023-05-30 09:33:14.830 UTC [85] LOG:  checkpoint complete: wrote 946 buffers (5.8%); 0 WAL file(s) added, 0 removed, 0 recycled; write=1.97
4 s, sync=0.005 s, total=2.010 s; sync files=260, longest=0.003 s, average=0.001 s; distance=4444 kB, estimate=4444 kB
docker-db-1 | .2023-05-30 09:33:14.919 UTC [84] LOG:  database system is shut down
docker-db-1 | done
docker-db-1 | server stopped
docker-db-1 |
docker-db-1 | PostgreSQL init process complete; ready for start up.
docker-db-1 |
docker-db-1 | 2023-05-30 09:33:15.089 UTC [1] LOG:  starting PostgreSQL 15.3 (Debian 15.3-1.pgdg110+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 10.2
.1-6) 10.2.1 20211010, 64-bit
docker-db-1 | 2023-05-30 09:33:15.090 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
docker-db-1 | 2023-05-30 09:33:15.090 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
docker-db-1 | 2023-05-30 09:33:15.098 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
docker-db-1 | 2023-05-30 09:33:15.213 UTC [1001] LOG:  database system was shut down at 2023-05-30 09:33:14 UTC
docker-db-1 | 2023-05-30 09:33:15.302 UTC [1] LOG:  database system is ready to accept connections
docker-web-1 | 30-May-2023 09:33:15.447 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server version name: Apache Tomcat/10.1.9
docker-web-1 | 30-May-2023 09:33:15.456 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server built: May 9 2023 14:30:05 UTC
docker-web-1 | 30-May-2023 09:33:15.459 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server version number: 10.1.9.0
docker-web-1 | 30-May-2023 09:33:15.460 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Name: Linux
docker-web-1 | 30-May-2023 09:33:15.461 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Version: 5.15.49-linuxkit
docker-web-1 | 30-May-2023 09:33:15.466 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Architecture: amd64
docker-web-1 | 30-May-2023 09:33:15.466 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Java Home: /opt/java/openjdk
docker-web-1 | 30-May-2023 09:33:15.472 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log JVM Version: 17.0.7+7
docker-web-1 | 30-May-2023 09:33:15.473 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log JVM Vendor: Eclipse Adoptium
docker-web-1 | 30-May-2023 09:33:15.473 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log CATALINA_BASE: /usr/local/tomcat
docker-web-1 | 30-May-2023 09:33:15.474 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log CATALINA_HOME: /usr/local/tomcat
docker-web-1 | 30-May-2023 09:33:15.503 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.config.
file=/usr/local/tomcat/conf/logging.properties
docker-web-1 | 30-May-2023 09:33:15.504 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.manager
=org.apache.juli.ClassLoaderLogManager
docker-web-1 | 30-May-2023 09:33:15.506 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djdk.tls.ephemeralDHKeySiz
e=2048
docker-web-1 | 30-May-2023 09:33:15.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.protocol.handler.pk

```

Connecting to `localhost:8080/crane` we can interact with the web application



Containerize your web application

To containerize your group project you have to:

- Install docker on your machine;
- Modify the docker-compose.yml file provided for crane project (the parts to be modified are those highlighted in green);
- Place this file in the same folder of your war file generated with maven;

services:

```
web:
  image: tomcat:10
  ports:
    - "8080:8080"
  depends_on:
    db:
      condition: service_healthy
  volumes:
    - ./crane.war:/usr/local/tomcat/webapps/crane.war

db:
  image: postgres
  ports:
    - "5432:5432"
  environment:
    - POSTGRES_PASSWORD=postgres
    - POSTGRES_USER=postgres
  volumes:
    - ./crane.sql:/docker-entrypoint-initdb.d/init.sql
    - ./data/db:/var/lib/postgresql/data
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U postgres" ]
    interval: 5s
    timeout: 10s
    retries: 50
```

Tomcat version to be modified if needed.

This should not be a problem if you do not have other services running on port 8080

The name of the war file depends on you application.

This should not be a problem if you do not have other services running on port 5432, otherwise put 5433:5432

The .sql file contains all the code to create and populate the database

Containerize your web application

To manage, run, stop, remove your container and have a clear overview of what is going on you can download Docker Desktop.

Open a new terminal, check that the docker daemon is alive and then type:

`docker-compose up` → This will create and run a new container.

`docker-compose down` → stops the container: your web application will not be visible anymore

`Docker ps` → list the RUNNING containers

`Docker ps -a` → list all the containers (also those exited with some error codes)

Containerize your web application

To interact with the database you can use psql. To access postgresql database:

`docker ps` → list the RUNNING containers. Select the name of the postgres container, in my case, `docker-db-1`

`docker exec docker-db-1 psql -U postgres` → psql command line – replace `docker-db-1` with your postgres container name